Events: Lessons learnt from building an Enterprise Data Bus

Swissport: Stefan Brock, Enterprise Architecture
sqooba: Markus Ehrenmann, Co-Creation
June 2019

Agenda

Stefan Brock
Head Enterprise Architecture
Swissport

Enterprise Architecture

Markus Ehrenmann
Co-Creator/ Architect
Sqooba

Realization Phase for Events

1. Introduction: Swissport and Architecture preconditions
2. Business Needs and Approach
3. Target Architecture, Processes, Lessons learned
4. Framing: Events and concurrent tech solutions
5. Context: Maturity, conditions, Technical Architecture
6. Lessons learned in Realization phase

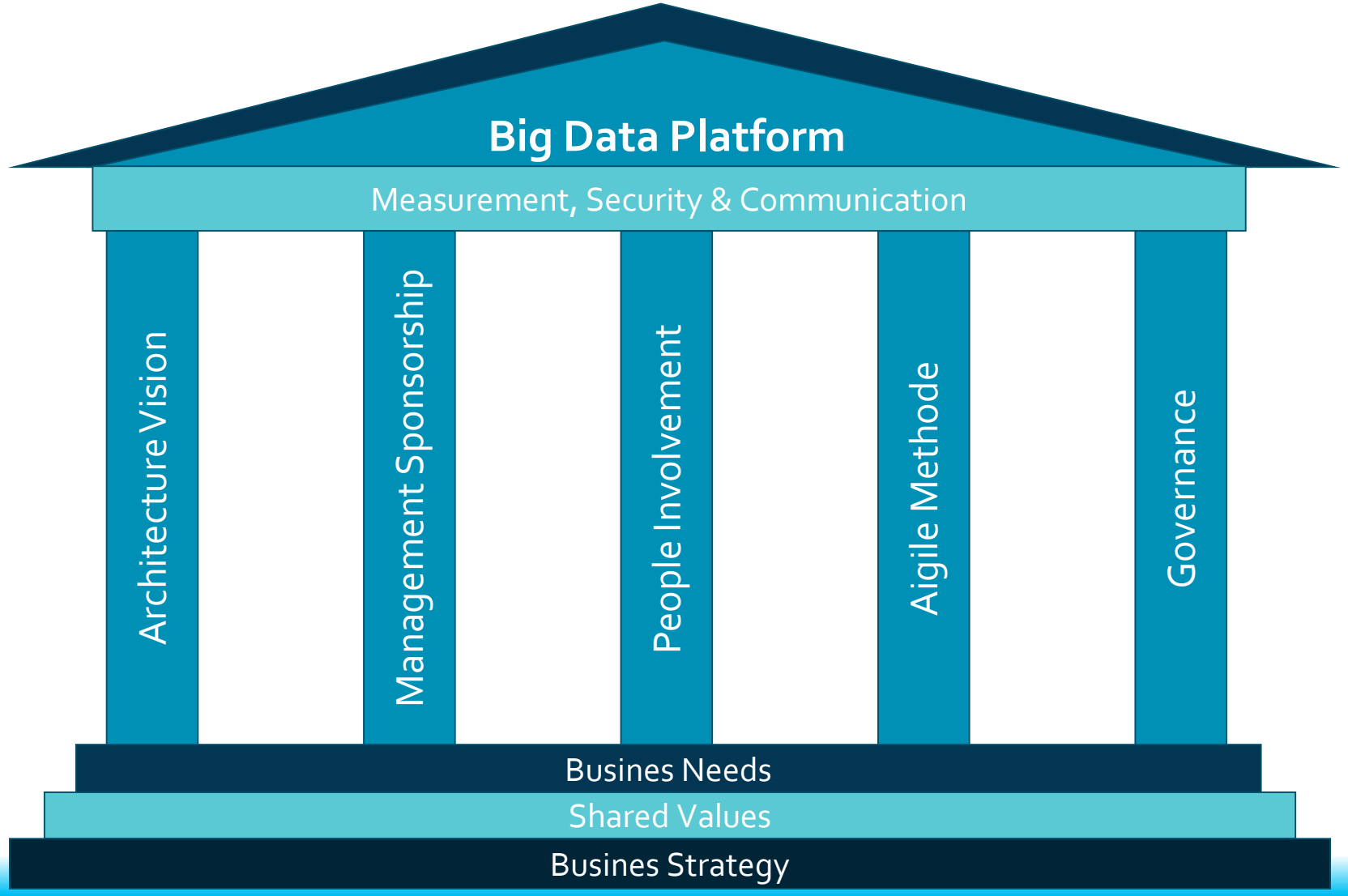
Agenda

Stefan Brock
Head Enterprise Architecture
Swissport

Enterprise Architecture

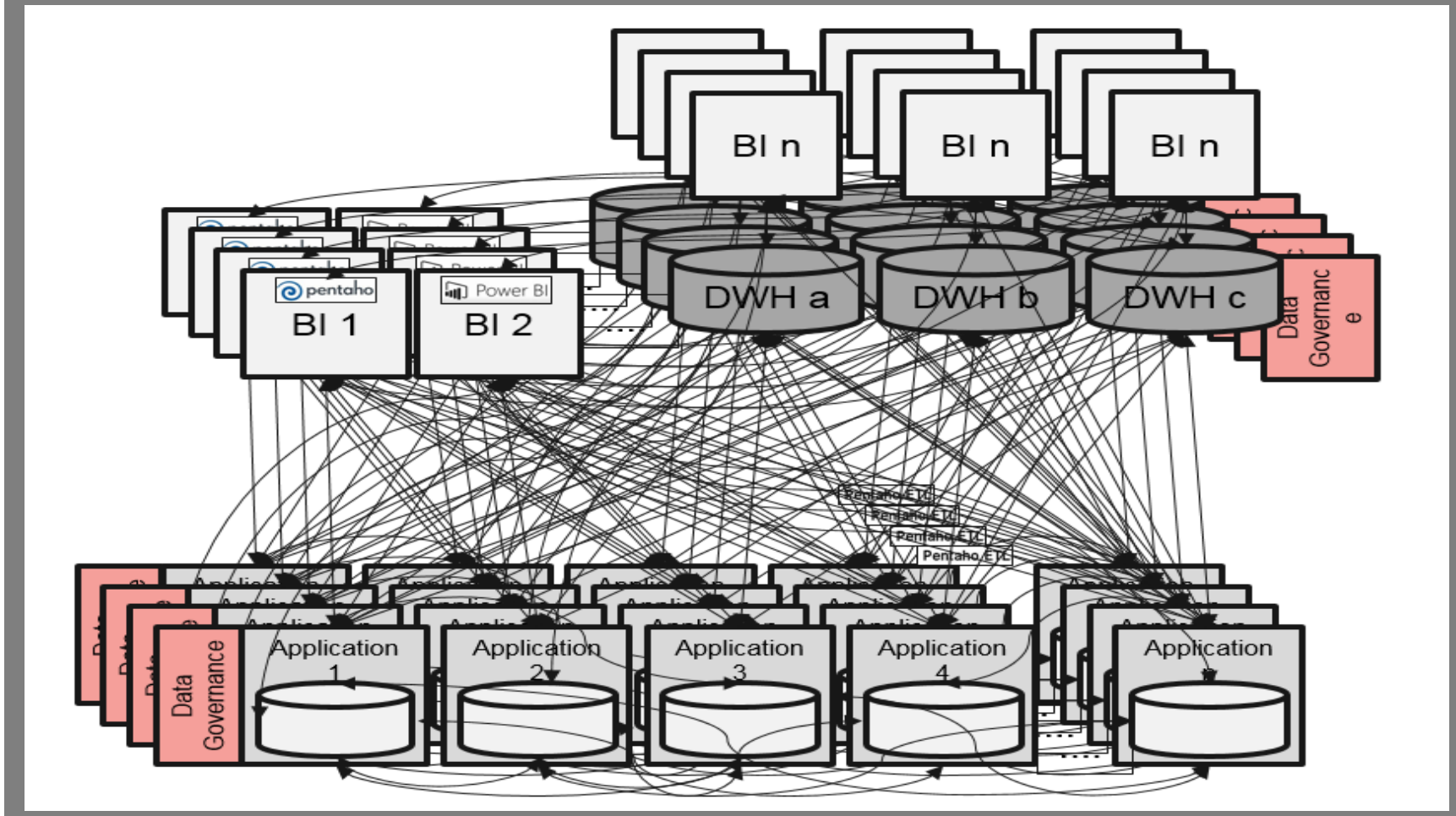
1. Introduction: Swissport and Architecture preconditions
2. Business Needs and Approach
3. Target Architecture

Big Data Architecture: Preconditions



Today's «Hairball» Architecture

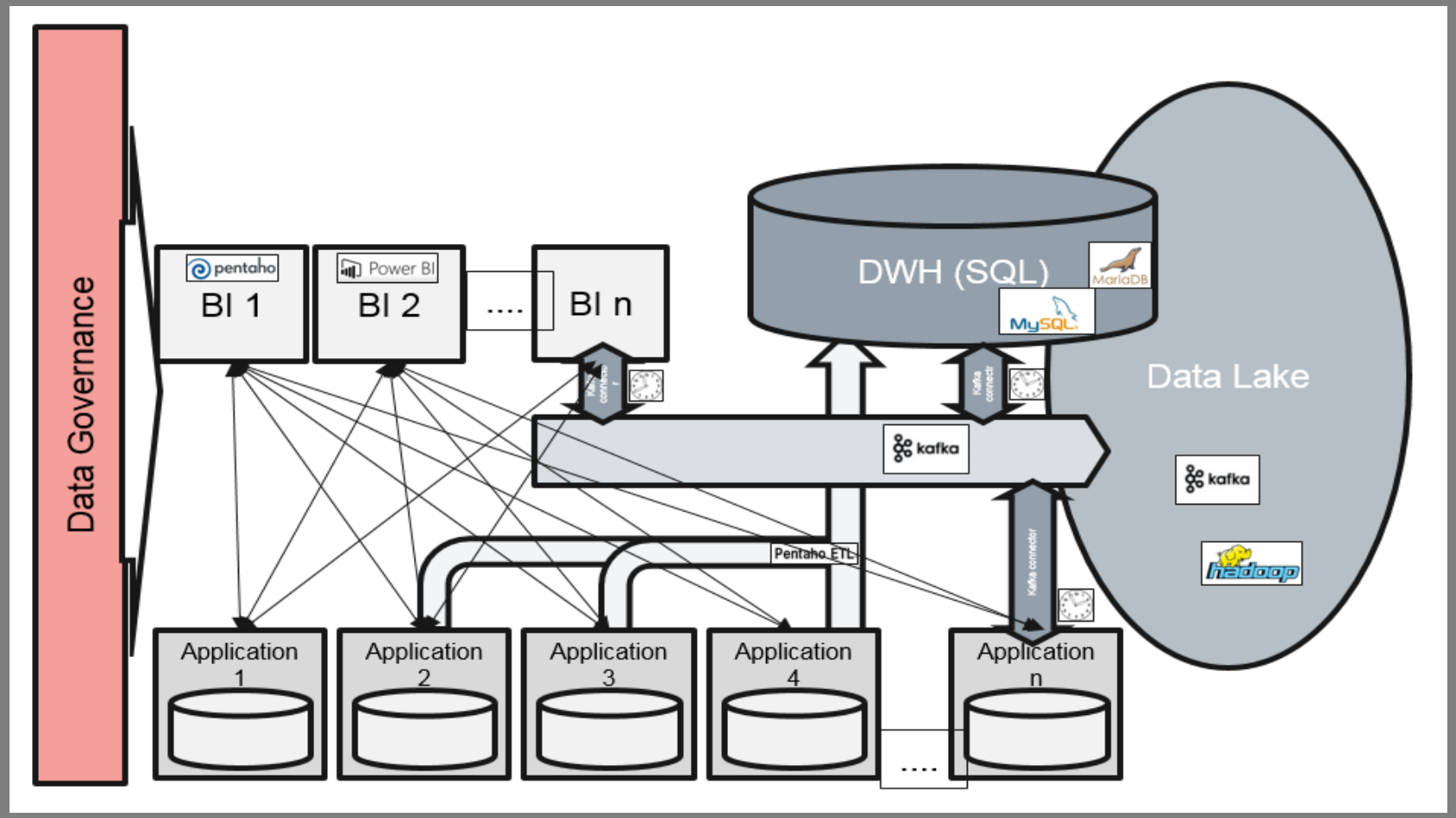
Todays IT "Hairball" Architecture



- IT Silos with thousends of interfaces
- Low protection
- No monitoring
- Difficult to operate
- Inconsistent data
- Complex and extensive interfaces to develop
- Frequent changes on interfaces

The Way forward

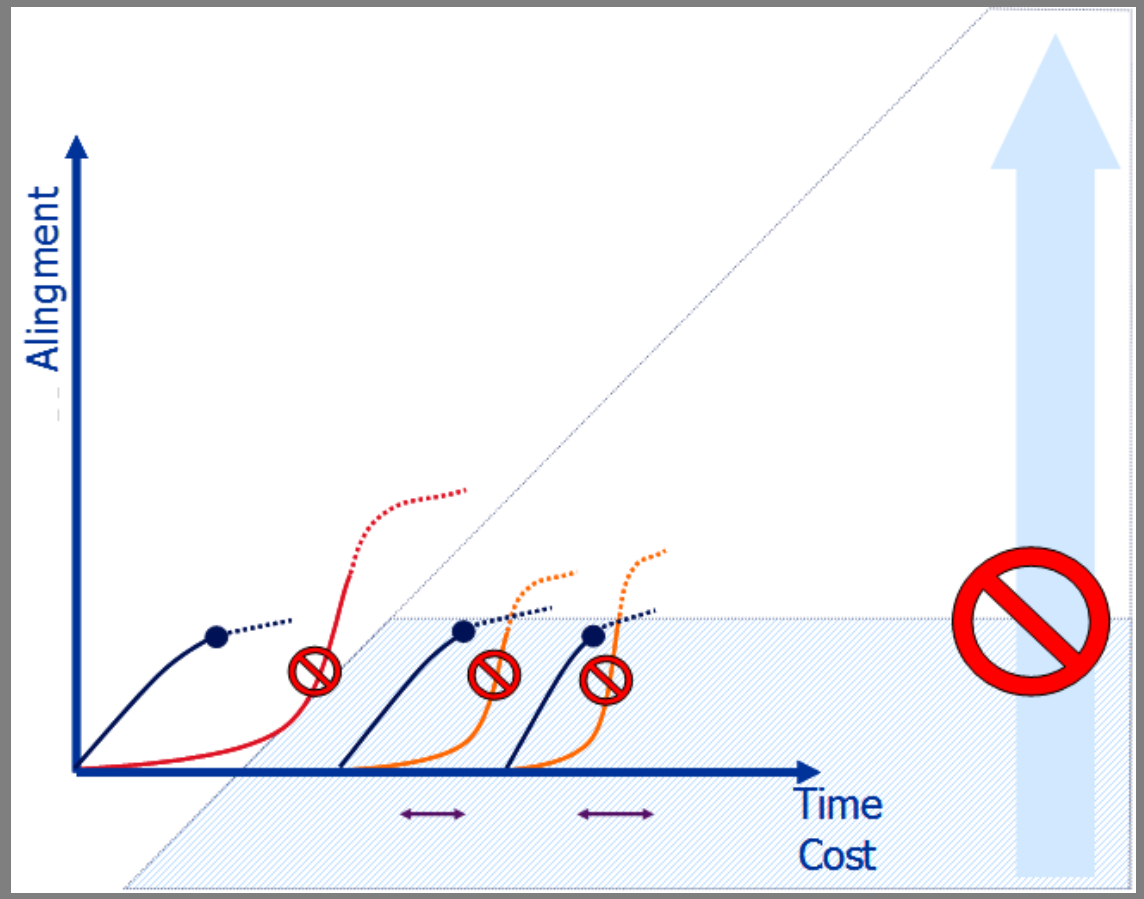
The Way Forward

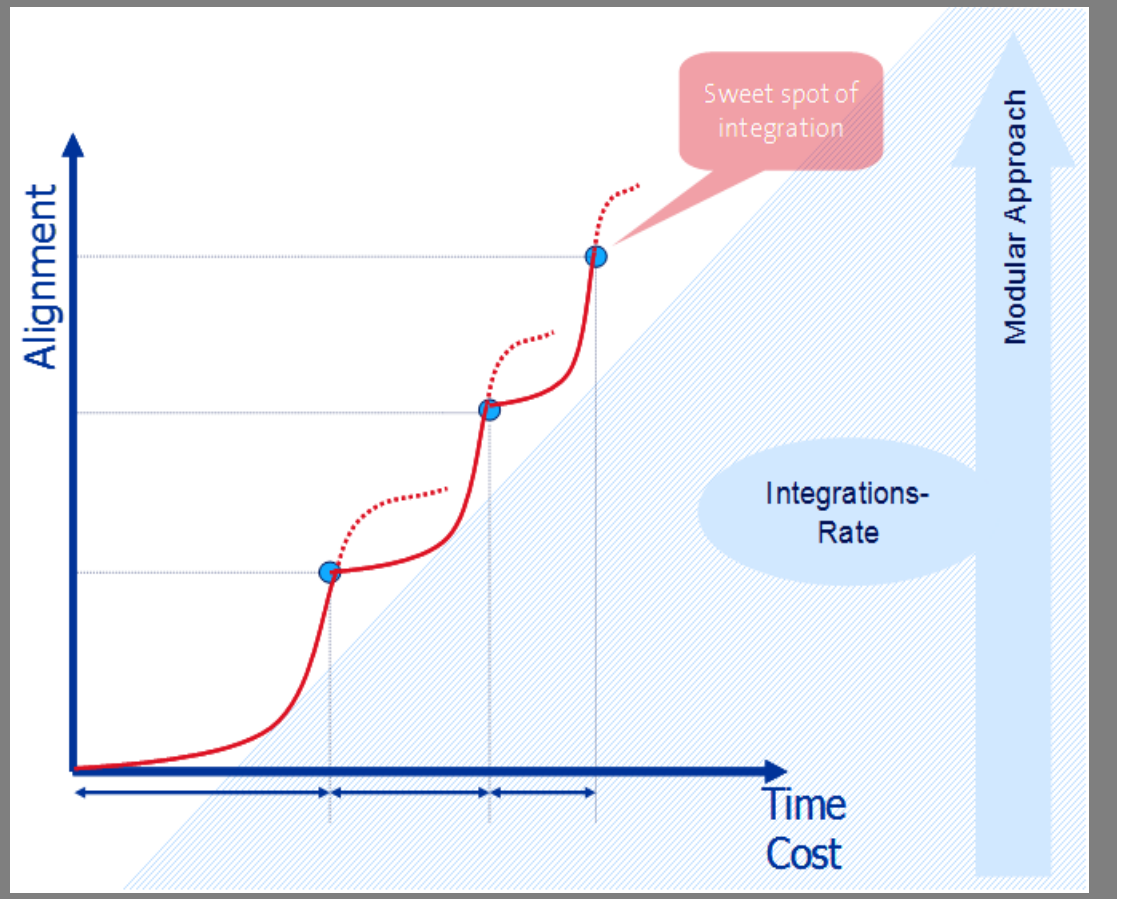


- June IT Architecture and Apps aligned on the design prinicple to connect all apps using Kafka
- December 4th 2018: Enterprise Data Bus (Kafka) gone live
- December 10th: P2P PoC shows ability to connect SAP, Ivalua and Maximo within few days

Initial savings

First Step: Commercial Value

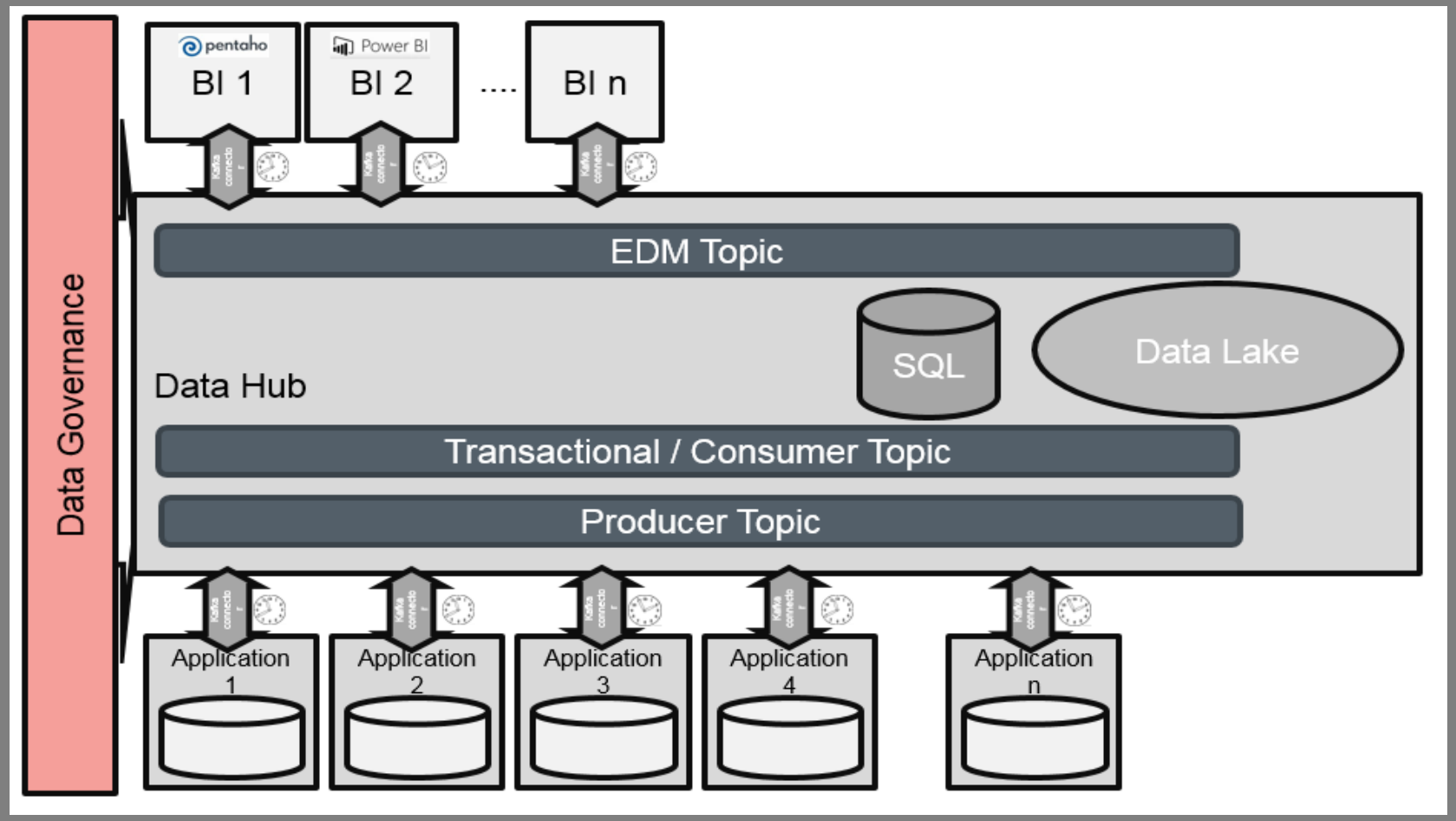




Source_ Sqooba, February 1st 2019

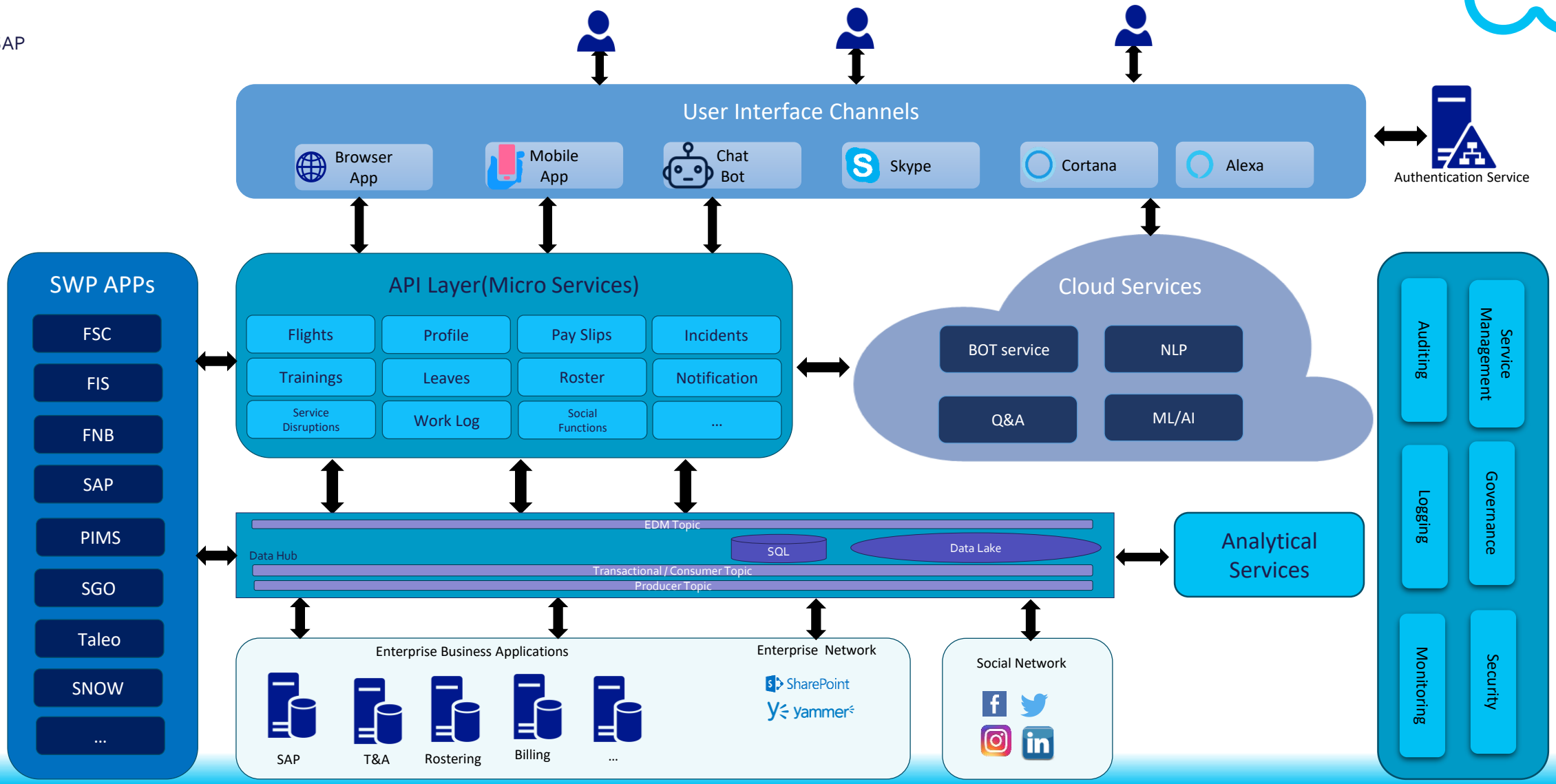
Future «Data Hub» Architecture

Future "Data Hub" Architecture/ Data as a Service

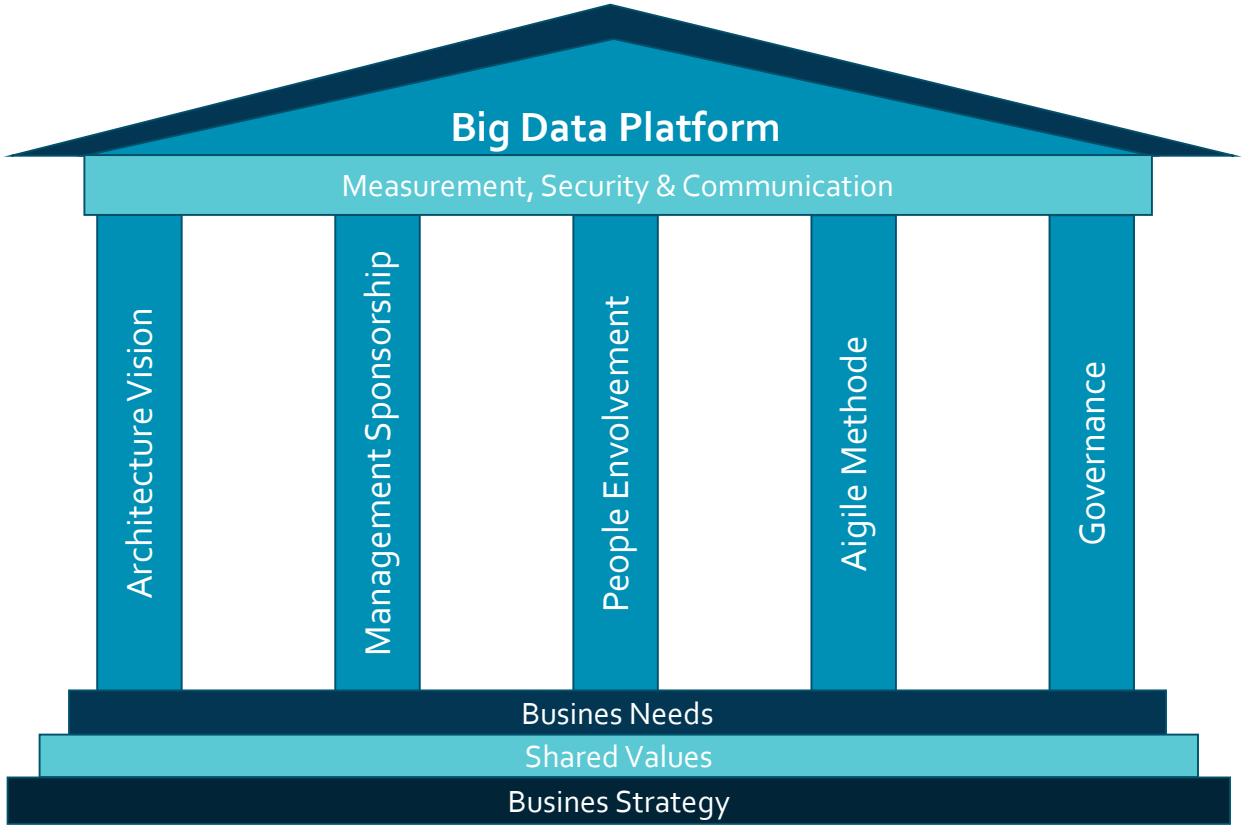


- Standard interface to the Kafka to reduce cost of connection be design, re-use and approach
- Enable E2E monitoring and operation
- Architecture ensures data documentation, protection, access control and availability
- Data quality increase by transparency
- Data as a service will be enabled

Big Data Architecture: Architecture Vision



Lessons learned
Enterprise Architecture



Enables:
- increased data quality
- data protection
- new data fueled services

Needs:
- consequence to follow on
- strategy and architecture
- no bypasses due to ease of integration, time pressure or technical complaints
- top management sponsorship for set up - and over at least 3 years after start
- clear architecture guidance

Gain contributors and champions to the approach!
«Embrace Agility» !
Celebrate!

Agenda

Markus Ehrenmann
Co-Creator/ Architect
Sqooba

Realization Phase for Events

4 Framing: Events and concurrent tech solutions

5 Context: Maturity, conditions, Technical Architecture

6 Lessons learned in Realization phase

Framing 1: What is an Event? Characteristics

event noun

\ i-ˈvent \

Merriam-Webster

Definition of *event*

1 a : something that happens : OCCURRENCE

b : a noteworthy happening

c : a social occasion or activity

d : an adverse or damaging medical occurrence
// a heart attack or other cardiac *event*

2 : any of the contests in a program of sports

3 a : a postulated outcome, condition, or eventuality
// in the *event* that I am not there, call the house

b : the final outcome or determination of a legal action

c *archaic* : OUTCOME

4 : the fundamental entity of observed physical reality represented by a point designated by three coordinates of place and one of time in the space-time continuum postulated by the theory of relativity

5 : a subset of the possible outcomes of an experiment

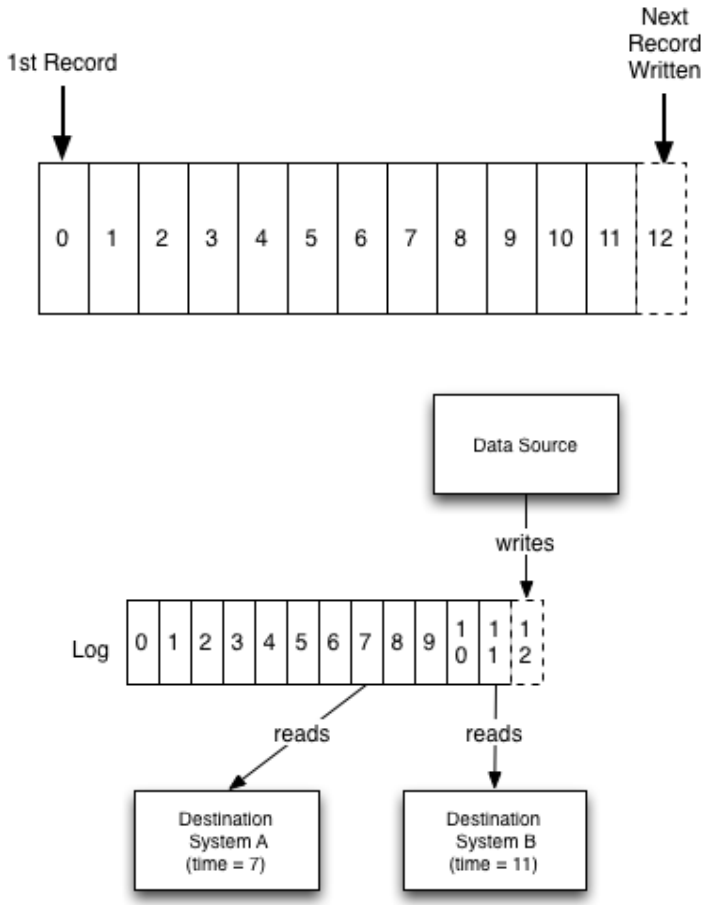
Something that **happens at a given time** and that **we know of (fact + notification).** Different **types of events** may occur. And we don't know when/ if occurences stop.

Relevant characteristic: ***unbounded* data,** continuously updated: no (known) end to new arrivals

Immutability: an event description is an irreversible fact

Distinguish **time of occurence,** of **ingestion** and time of **processing**

Framing 2: How to handle Events?
Order and the replayable Log



Events are ordered entities – by time
Appropriate data structure for Events: the Log

Stream Processing
Process records as they arrive
(as opposed to batch processing: process a collection of records at a certain point in time)

Pub-Sub
Several services may be interested to get their notifications
Buffer and distribute in order to decouple producers and consumers

➔ Technical Solution: Replayable log
1) Central repository of datasets for whoever needs to consume them
2) Reactions to events are made easy with a toolset for manipulation

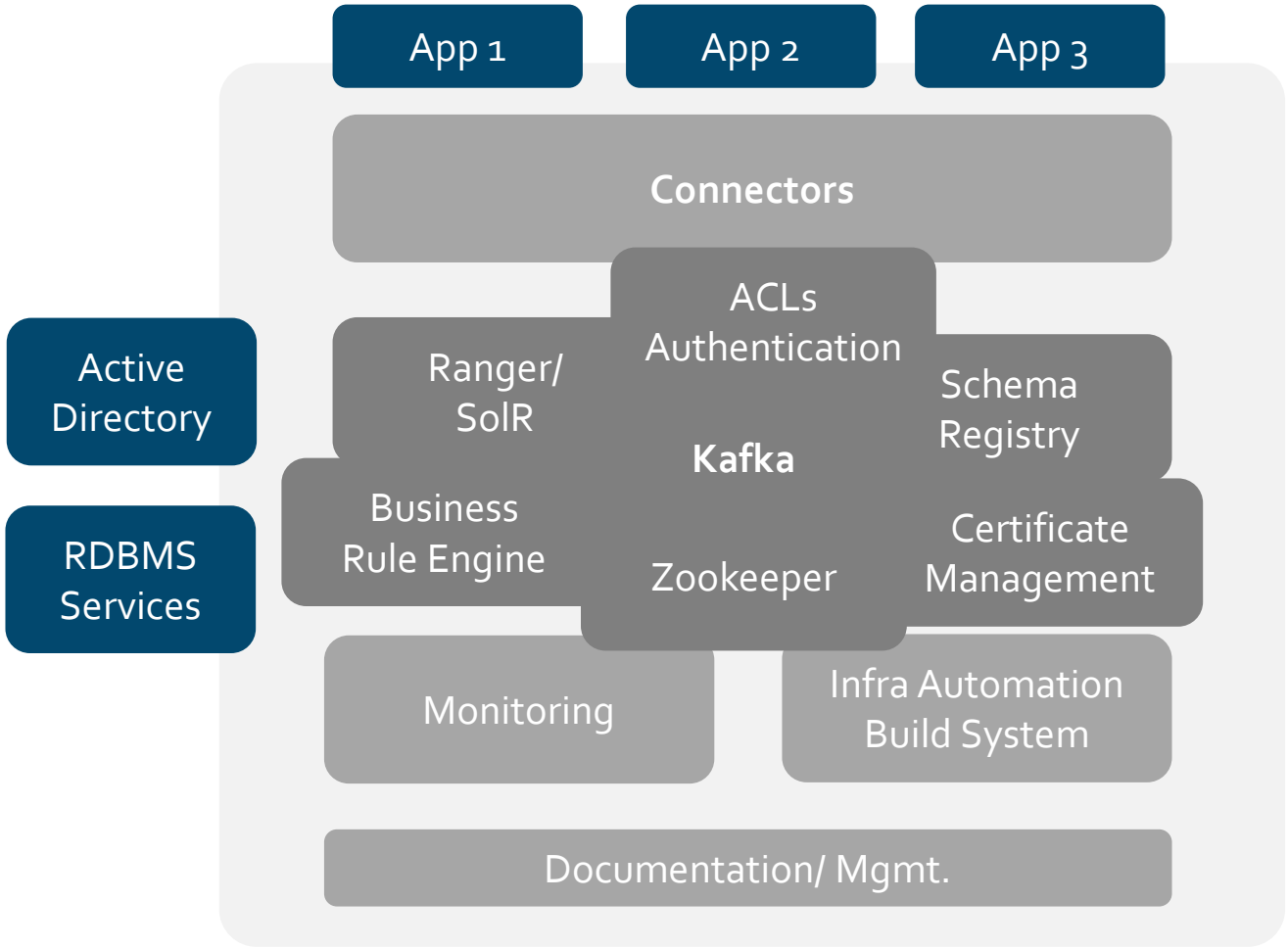
Center: **Apache Kafka**, Pulsar, nats.io, Wallaro
Frameworks for distributed processing around center: **Kstreams/ KSQL**, Beam, Flink, Spark…

https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying

10 Sprints

bi-weekly, 1 product owner, 4 engineers

Technical Architecture & Guiding Principles



Databus Principles

Created by Unknown User (markus), last modified by Bonello, Charles on 2019-02-07

Following principle 5 of the Application Architecture Guidelines "Standardization" – quote: "Use standards where possible - reduce customizing to the max" we limit the space in which connectors to the databus will be implemented.

- 3rd party providers and RESTful API
- Application and Topic content documentation
- Approach for external service providers
- Central Code Repository
- Certificates
- Connector Technologies
- Consolidated and Complete Records
- Consumer of Sensitive Data
- Data Format and Data Types
- Data Integration
- Handling Batches
- Handling Schemas
- Handling Sensitive Data in Topics
- Infrastructure Centralization
- Integration on central IT side
- Max 2 Connector versions in operation
- Message production
- Monitor
- Retention Period
- Scheduler
- Test Driven Development
- Topic and Kafka Configuration

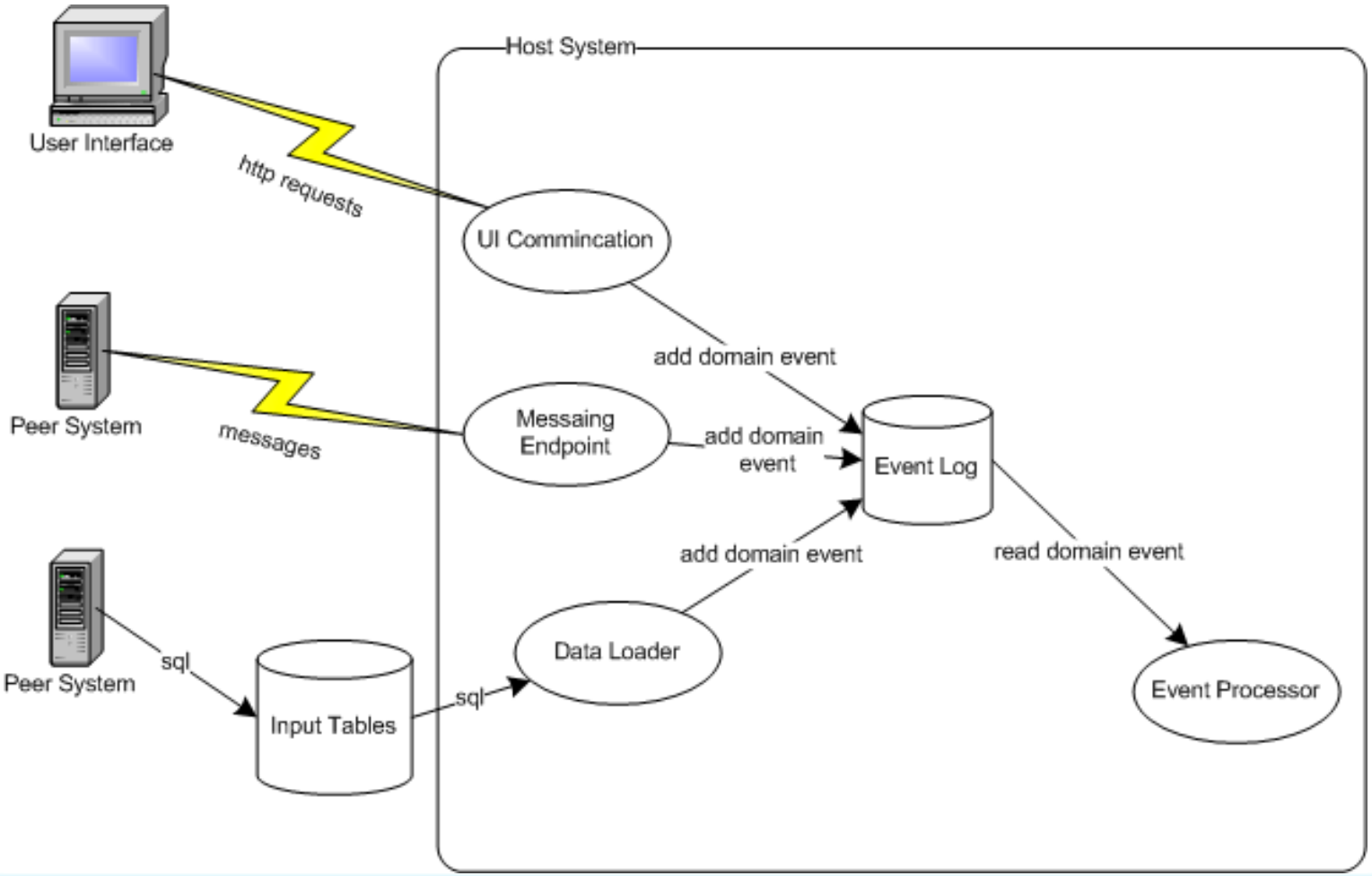
Observations
Reflections

1. Event Sourcing and Transformation
2. Data Governance
3. Centralization/ decentralization:
   the „ESB antipattern“ discussion

How to handle Events?
Content – Martin Fowler's Domain Events

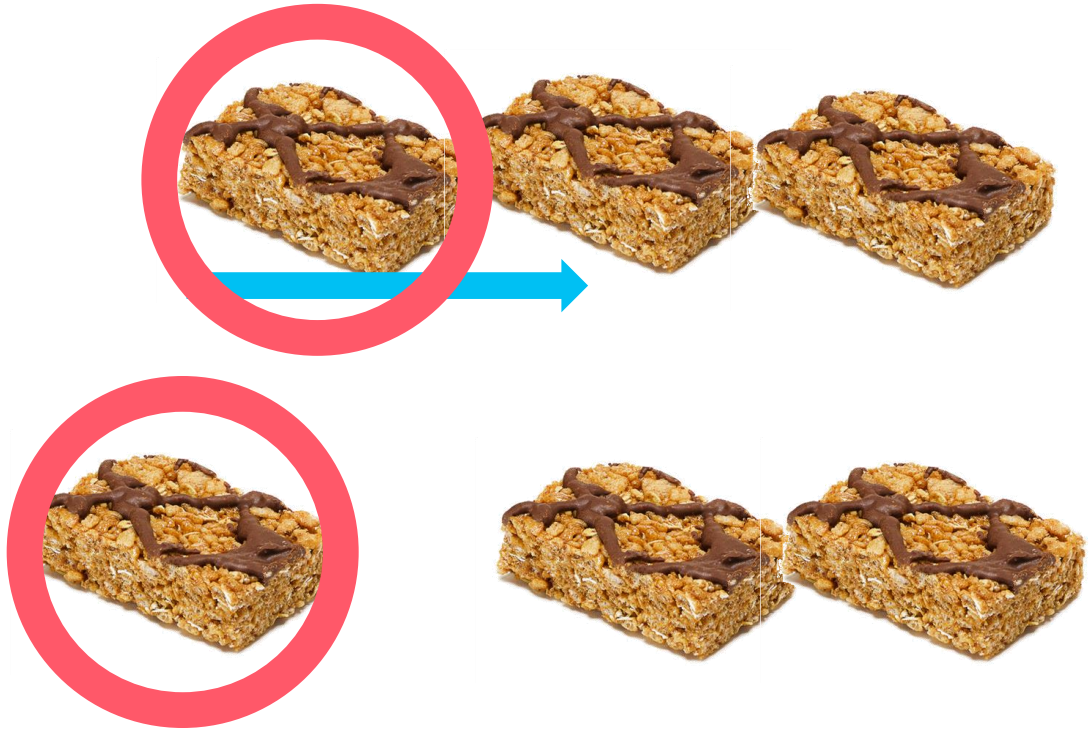
"Things happen. **Not all of them are interesting**, some may be worth recording but don't provoke a reaction.

The most interesting ones cause a reaction. Many systems need to react to interesting events.
Often you need to know why a system reacts in the way it did."



https://martinfowler.com/eaaDev/DomainEvent.html

How do Events look like (o)
Message Production



Not everyone supports the Streaming paradigm
There are applications that support batch only

How do Events look like (1)

Schemas as Contracts

https://www.confluent.io/blog/schemas-contracts-compatibility

How do Events look like (2)

Domain Event Sourcing is Connector Technology dependent!

What we actually want but rarely get

What we can provide fastest

What technology partners support the most

What specific technologies require

Statement:

As programming languages for *the producers*, we choose in descending order

1. Application-level events with a direct Kafka connector (if provider supports that)
2. CDC, technology: Debezium
3. Query based: Kafka Connect
4. Apache NiFi
5. edbic

All code owned by Swissport needs to be written in either of these languages/ libraries/ tools.

Consumers can be written in any language as long as the provider of the consumer supports it.

How Events *should* look like
And where we want to go

Consolidated and Complete Records

Created by Bonello, Charles, last modified by Sirre, Robert on 2019-04-09

Statement:

A connector should send consolidated records where appropriate. Consolidated records are:

- pre-joined records
- filtered for payload, excluding application internal management data
- deletions should be visible

Completeness

If possible, a connector should also send the complete set of attributes of the event/ entity, not only the ones required for a particular use case. O
messages.

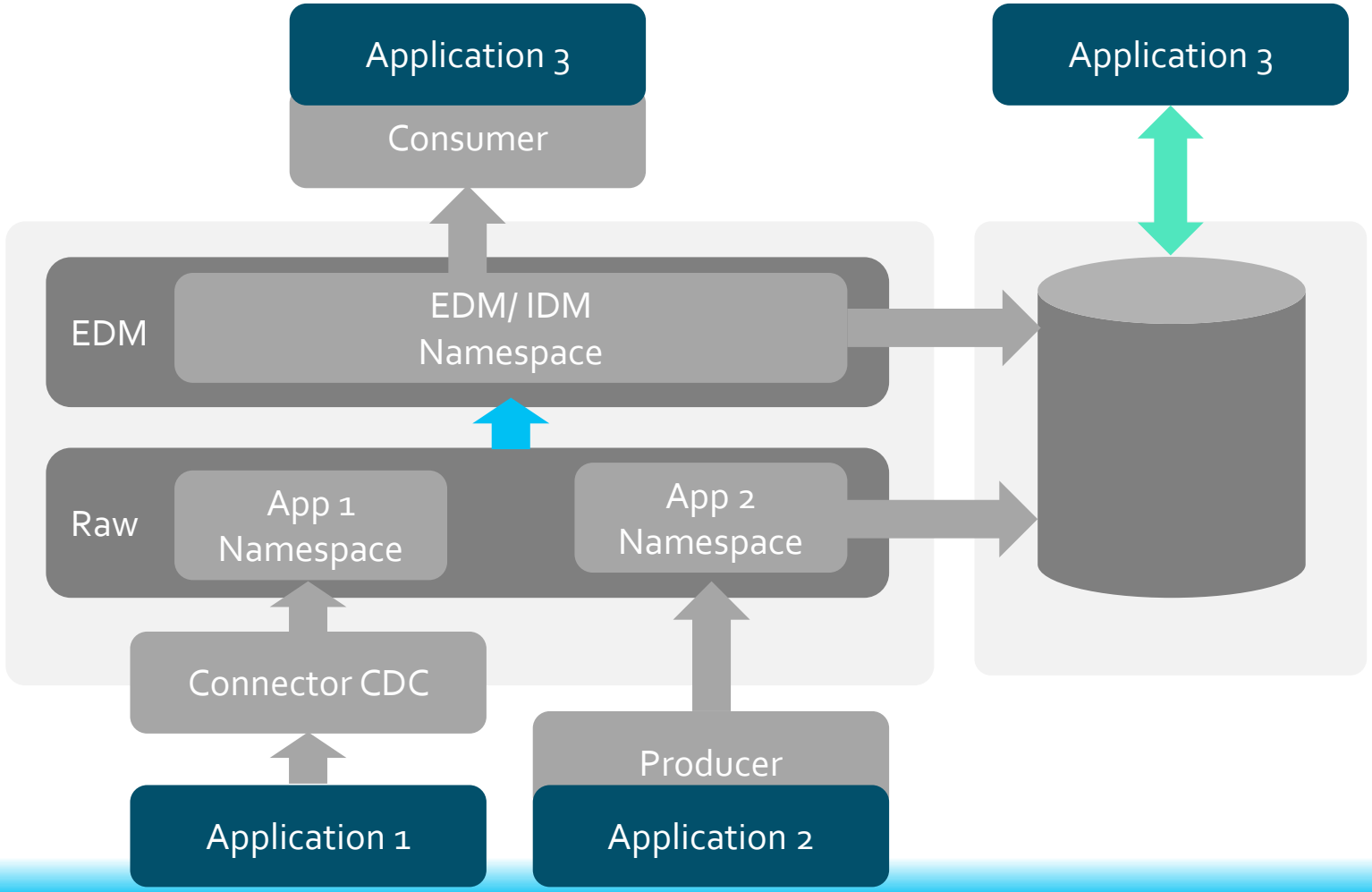
Technology: Apache NiFi or Kafka Connect which can also be leveraged on top of views: In order to increase the performance of the connector to
having a timestamp column storing the last modified date and a strictly growing PK where possible. The easiest approach to deletion handling for

Command-Query-Responsibility Separation (CQRS)

Separate Event types:
raw events, EDM/ IDM

Build EDM events with predefined queries with stateless/ stateful stream processing where possible

Use batch approaches on separately persisted data for complex analytical queries



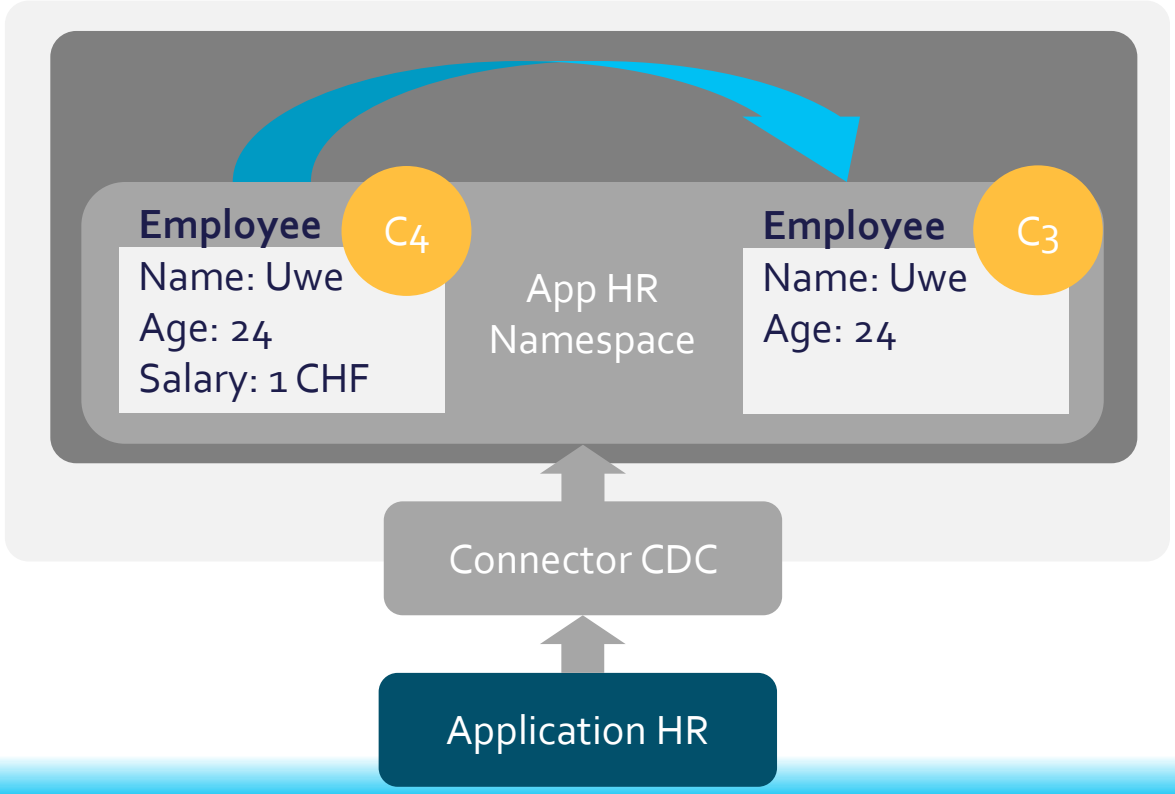
Data Governance
Process and Maturity Model

- Availability
- Documentation
- Modeling
- Classification
- Access

Statement:
ALL data on the Kafka bus starts life as unreachable by any consumer process.
Consideration:
All data that is produced on the Kafka bus whether it is initiated by a human or an automatic process MUST be unreachable by any consumer process until it is explicitly granted access. Granting access will be a multi stage process that requires producer and data detail.
Rationale:
Swissport creates and manages a vast amount of data on a daily basis. The Enterprise Architecture Team, working with Legal and Security Teams, is committed to ensuring that data is secured and access is only granted to the right individuals. There are a number of 'gates' that producer data needs to go through before it is available for consumption. This is the first one.
Exceptions:
Exceptions in current setup: • InforBI - only consumer is DCM, has a role base access concept

Data Access
Role-Based Access Control

Topic-level access protection based on name spaces for production and based on content for consumption.



Responsibility Delegation.

Consumer of Sensitive Data

Created by Ehrenmann, Markus (EXT) on 2019-03-21

Statement:

- Any consumer of sensitive data (classification level S3, S4) must
  - use the data only for a defined and agreed purpose (sign-off and documentation
  - prove data protection at the same level as EDB (encrypted channels and encrypti logging)

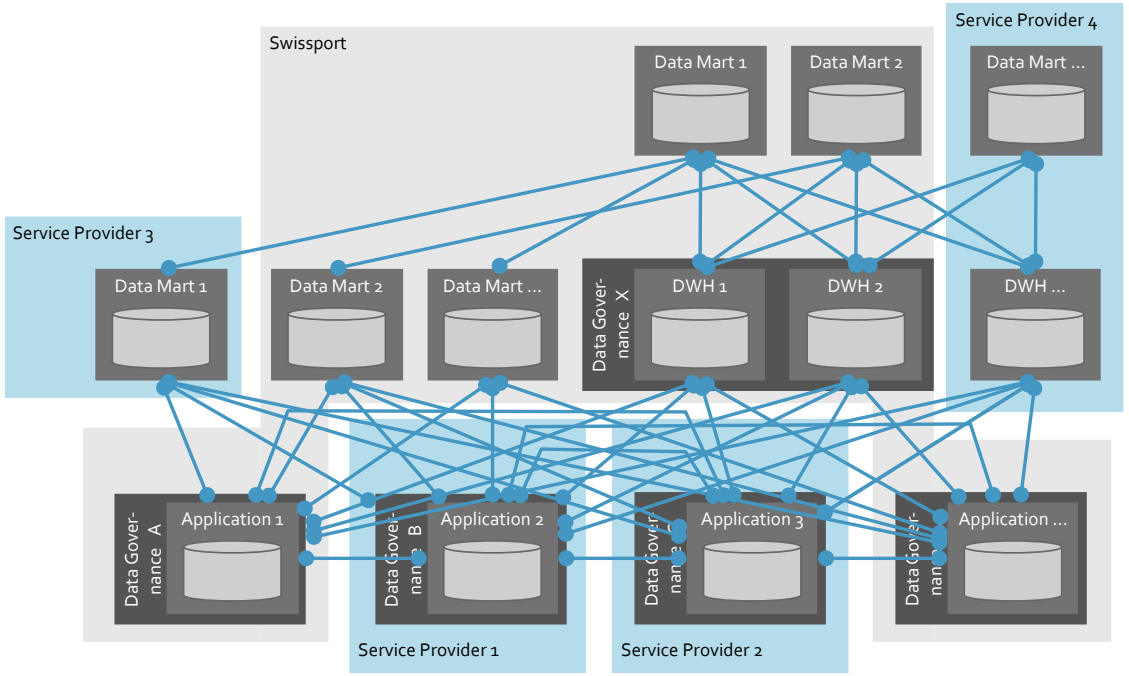
Consideration:

- Security and Compliance

Rationale:

- Security and Compliance Requirements must be delegated and ensured when publish

Self-Service versus Centralization
One after the other



- Blue: applications with data silos, typically outsourced engineering, no Swissport-internal know-how on data content
- Transversal analytics requires centralization of data understanding

Infrastructure Centralization

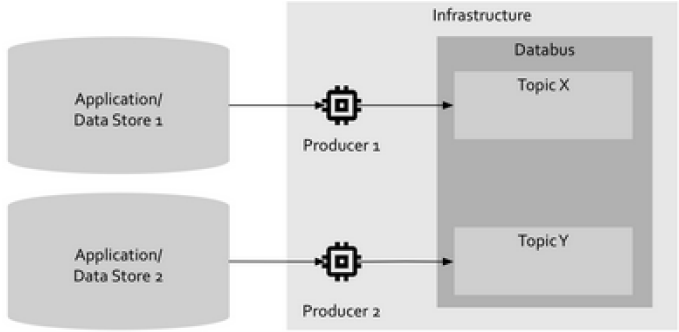
Created by Bonello, Charles on 2018-11-29

Statement:

Connector jobs need ressources to run on. These ressources need to be centralized onto common infrastructure. Example: Debezium, KConnect.

Consideration:

Each connector needs a machine to run on. In case it cannot be put onto the system running the application or data



Rationale:

Any job– especially with potentially sensitive data – requires ressources, protection. In the case of the databus and it'

It is more efficient to do this in one central instance only.

https://www.thoughtworks.com/radar/techniques/recreating-esb-antipatterns-with-kafka

Questions?
Links & References